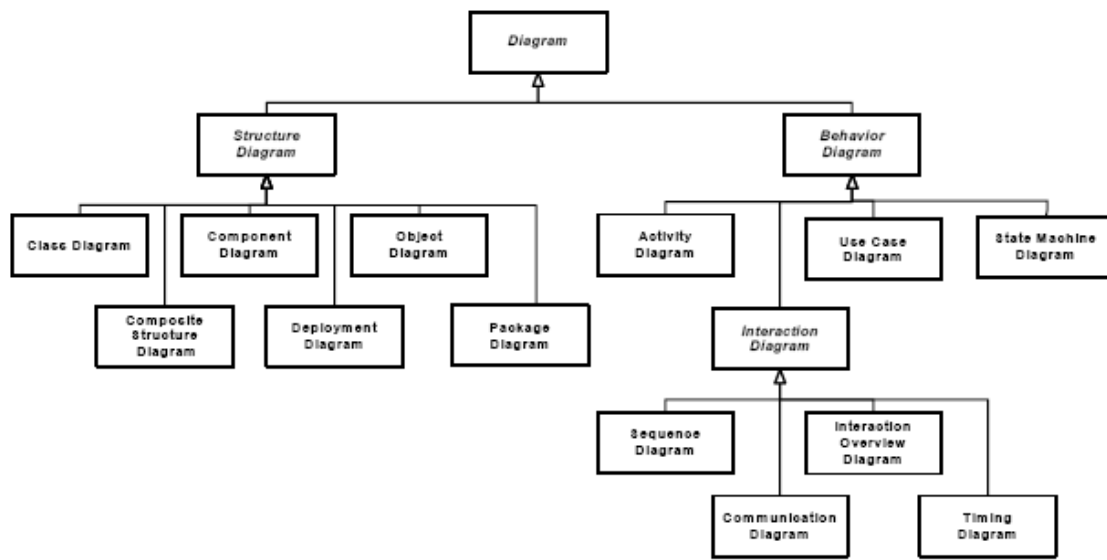




The taxonomy of structure and behavior diagram

Structure diagrams show the static structure of the objects in a system. That is, they depict those elements in a specification that are irrespective of time. The elements in a structure diagram represent the meaningful concepts of an application, and may include abstract, real-world and implementation concepts.

Structure diagrams do not show the details of dynamic behavior, which are illustrated by behavioral diagrams. However, they may show relationships to the behaviors of the classifiers exhibited in the structure diagrams.

Behavior diagrams show the dynamic behavior of the objects in a system, including their methods, collaborations, activities, and state histories. The dynamic behavior of a system can be described as a series of changes to the system over time.

This taxonomy provides a logical organization for the various major kinds of diagrams. However, it does not preclude mixing different kinds of diagram types, as one might do when one combines structural and behavioral elements (e.g., showing a state machine nested inside an internal structure). Consequently, the boundaries between the various kinds of diagram types are not strictly enforced.

Variations of structure diagrams often focus on particular structural aspects, such as relationships between packages, showing instance specifications, or relationships between classes. There are no strict boundaries between different variations; it is possible to display any element you normally display in a structure diagram in any variation.

Class diagram

The following nodes and edges are typically drawn in a class diagram:

- Association
- Aggregation
- Class
- Composition
- Dependency
- Generalization
- Interface
- InterfaceRealization
- Realization

Object diagram

The following nodes and edges are typically drawn in an object diagram:

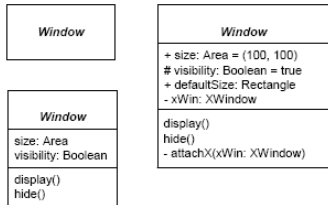
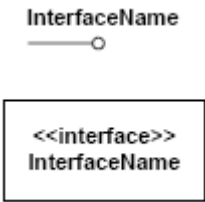
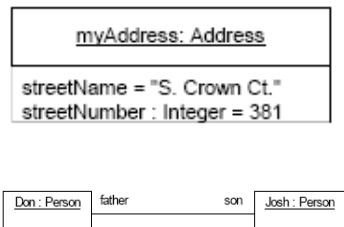
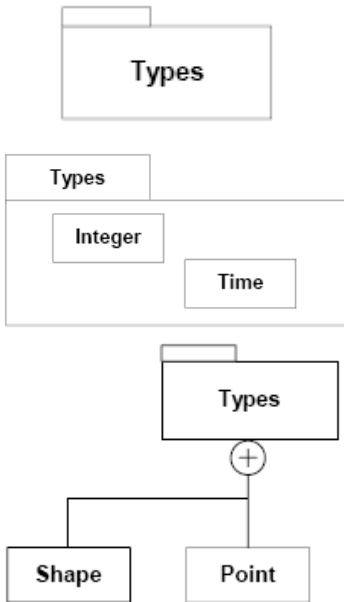
- InstanceSpecification
- Link (i.e., Association)

Package diagram

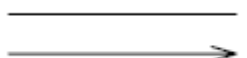
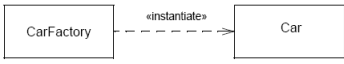
The following nodes and edges are typically drawn in a package diagram:

- Dependency
- Package
- PackageExtension
- PackageImport

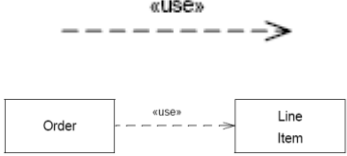
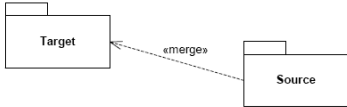
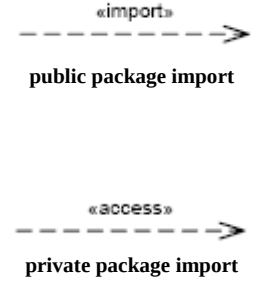
Graphic nodes included in structure diagrams

Node Type	Notation	Description
Class		Specifies a classification of objects, as well as, the features that characterize the structure and behavior of those objects.
Interface		An interface is a kind of classifier that represents a declaration of a set of coherent public features and obligations. An interface specifies a contract; any instance of a classifier that realizes the interface must fulfill that contract.
Instance Specification		Instances of any classifier can be shown by prefixing the classifier name by the instance name followed by a colon and underlining the complete name string. An instance specification whose classifier is an association describes a link of that association.
Package		A package is used to group elements, and provides a namespace for the grouped elements. It is also a packageable element that can be contained in other packages.

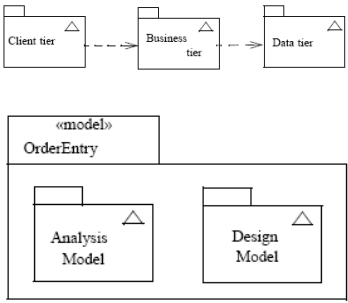
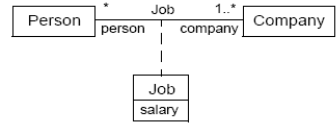
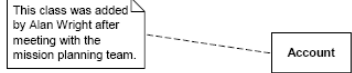
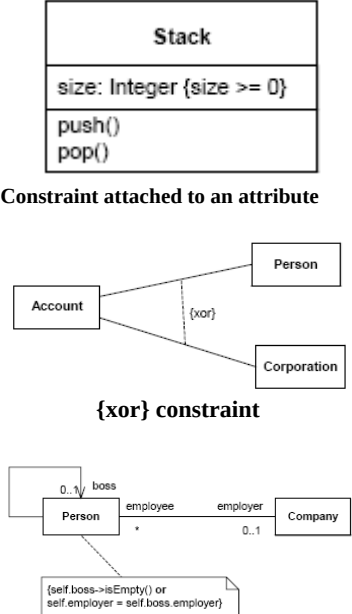
Graphic paths included in structure diagrams

Path Type	Notation	Description
Aggregation		An aggregation represents a whole/part relationship.
Association	 	An association specifies a semantic relationship that can occur between typed instances. An open arrowhead on the end of an association indicates the end is navigable. A small x on the end of an association indicates the end is not navigable. Notations that can be placed near the end of the line as follows: • name – name of association end • multiplicity • property string enclosed in curly braces ○ {ordered} to show that the end represents an ordered set. ○ {bag} to show that the end represents a collection that permits the same element to appear more than once. ○ {sequence} or {seq} to show that the end represents a sequence
Link		An instance of an association
Composition		Composite aggregation is a strong form of aggregation that requires a part instance be included in at most one composite at a time. If a composite is deleted, all of its parts are normally deleted with it. Deleting an element will also result in the deletion of all elements of the subgraph below that element.
Dependency	  instantiate dependency	A dependency is a relationship that signifies that a single or a set of model elements requires other model elements for their specification or implementation. The modification of the supplier may impact the client model elements.
Generalization		A generalization is a taxonomic relationship between a more general classifier and a more specific classifier. Generalization hierarchies must be directed and acyclical.

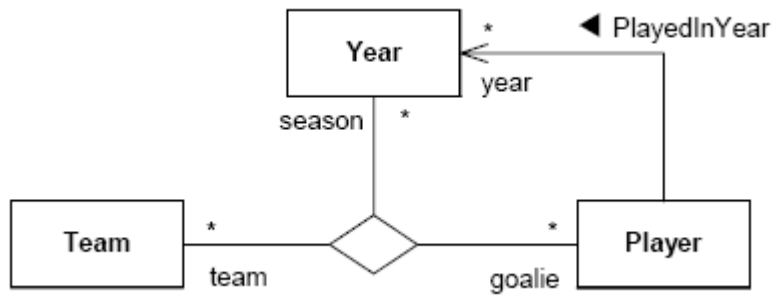
Class Diagram / Object Diagram / Package Diagram

Path Type	Notation	Description
Interface Realization		A specialized Realization relationship between a Classifier and an Interface. This relationship signifies that the realizing classifier conforms to the contract specified by the Interface. A classifier may implement a number of interfaces. The set of interfaces implemented by the classifier are its <i>provided</i> interfaces and signify the set of services the classifier offers to its clients.
Realization		Signifies that the client set of elements are an implementation of the supplier set, which serves as the specification.
Usage	 An Order class requires the Line Item class for its full implementation.	A usage is a relationship in which one element requires another element (or set of elements) for its full implementation or operation.
Package Merge		A package merge is a directed relationship between two packages that indicates that the contents of the two packages are to be combined. The owned elements of packages Target and Source are all incorporated into the namespace of package Source.
Package Import	 public package import private package import	A directed relationship between an importing namespace and a package, indicating that the importing namespace adds the names of the members of the package to its own namespace. If the PackageImport is public, the imported elements will be visible outside the package, while if it is private they will not.

Miscellaneous elements included in structure diagrams

Type	Notation	Description
Model		The Model construct is defined as a Package. It contains a (hierarchical) set of elements that together describe the physical system being modeled. A Model may also contain a set of elements that represents the environment of the system, typically Actors, together with their interrelationships, such as Associations and Dependencies.
Association Class		Defines a set of features that belong to the relationship itself and not to any of the classifiers
Comment		A comment is a textual annotation that can be attached to a set of elements. A comment gives the ability to attach various remarks to elements. A comment carries no semantic force, but may contain information that is useful to a modeler. The connection to each annotated element is shown by a separate dashed line.
Constraint	 Constraint attached to an attribute {xor} constraint Constraint in a note symbol	A constraint is a condition or restriction expressed in natural language text or in a machine readable language for the purpose of declaring some of the semantics of an element. One predefined language for writing constraints is OCL.

Class Diagram / Object Diagram / Package Diagram



The solid triangle indicates the order of reading: *Player PlayedInYear Year*.

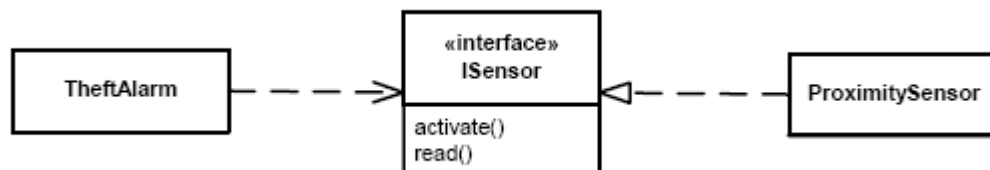


Qualified associations

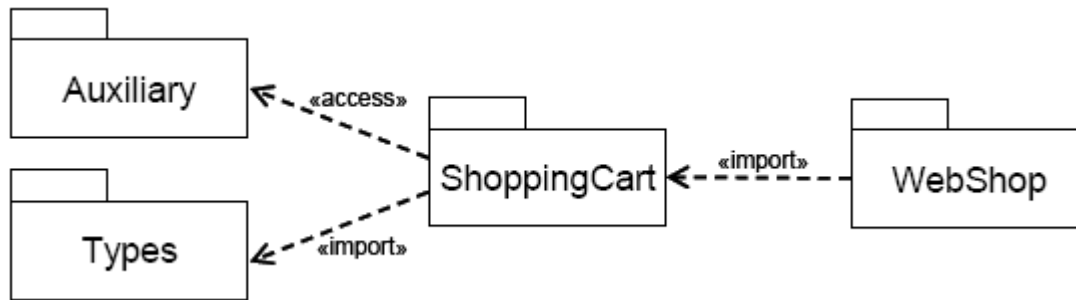


ISensor is the required interface of TheftAlarm

ISensor is the provided interface of ProximitySensor



Alternative notation



Examples of public and private package imports

The elements in Types are imported to ShoppingCart, and then further imported to WebShop. However, the elements of Auxiliary are only accessed from ShoppingCart, and cannot be referenced using unqualified names from WebShop.

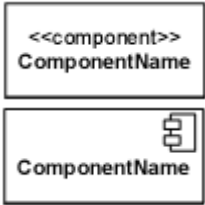
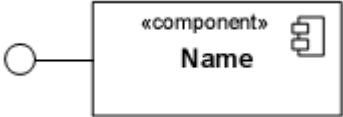
Component Diagram

A component can always be considered an autonomous unit within a system or subsystem. It has one or more provided and/or required interfaces (potentially exposed via ports), and its internals are hidden and inaccessible other than as provided by its interfaces. A component is encapsulated and its dependencies are designed such that it can be treated as independently as possible. As a result, components and subsystems can be flexibly reused and replaced by connecting (“wiring”) them together via their provided and required interfaces.

The following nodes and edges are typically drawn in a component diagram:

- Component
- Interface
- ComponentRealization, Interface Realization, Usage Dependencies
- Class
- Artifact
- Port

Graphic nodes included in component diagrams

Node Type	Notation	Description
Component		A component is a subtype of Class that provides for a Component having attributes and operations, and being able to participate in Associations and Generalizations. A component has a number of provided and required Interfaces, that form the basis for wiring components together, either using Dependencies, or by using Connectors. A component may be manifest by one or more artifacts, and in turn, that artifact may be deployed to its execution environment. A deployment specification may define values that parameterize the component’s execution. UML standard stereotypes that apply to Component: «subsystem» to model large-scale components «specification» and «realization» to model components with distinct specification and realization definitions, where one specification may have multiple realizations.
Component implements Interface		Provided interfaces describe the services that the instances of that classifier offer to their clients.

Component Diagram

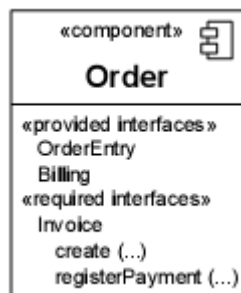
Node Type	Notation	Description
Component uses Interface		Required interfaces specify services that a classifier needs in order to perform its function and fulfill its own obligations to its clients.
Component has provided Port (typed by Interface)		The provided interfaces of a port characterize requests to the classifier that its environment may make through this port.
Component has required Port (typed by Interface)		The required interfaces of a port characterize the requests that may be made from the classifier to its environment through this port.
Component has complex Port (typed by provided and required Interfaces)		
Port	portName: ClassifierName Behavior port notation	Ports represent interaction points between a classifier and its environment. The interfaces associated with a port specify the nature of the interactions that may occur over a port. A service port is used to provide the published functionality of a classifier. A behavior port is indicated by a port being connected through a line to a small state symbol drawn inside the symbol representing the containing classifier.

Graphic paths included in component diagrams

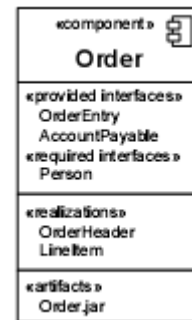
Path Type	Notation	Description
Connector		See Assembly Connector and Delegation Connector
Assembly Connector		An assembly connector is a connector between two components that defines that one component provides the services that another component requires.

Component Diagram

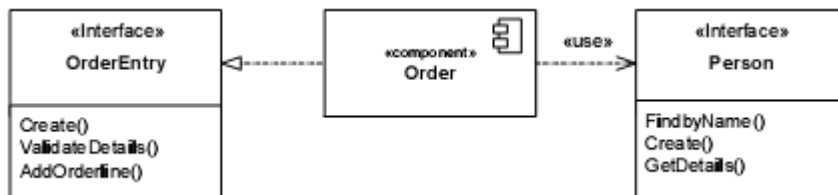
Path Type	Notation	Description
Delegation Connector		A delegation connector is a connector that links the external contract of a component (as specified by its ports) to the internal realization of that behavior by the component's parts. It represents the forwarding of signals (operation requests and events): a signal that arrives at a Port or Component Interface that has a delegation connector to a part or to another Port will be passed on to that target for handling.
Dependency		Same as for Class Diagram



Black box notation showing a listing of the properties of a component

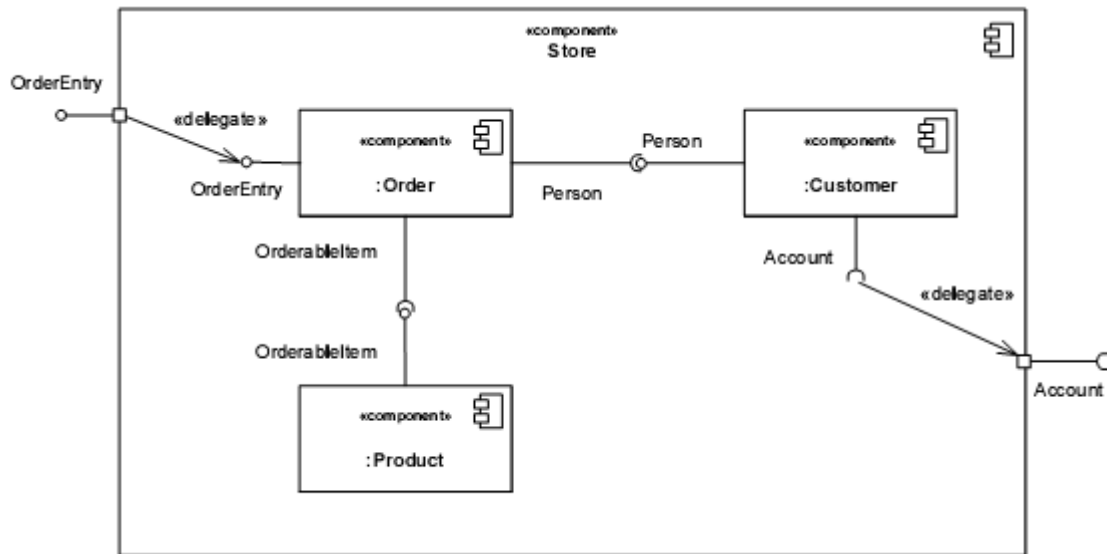


A white-box representation of a component

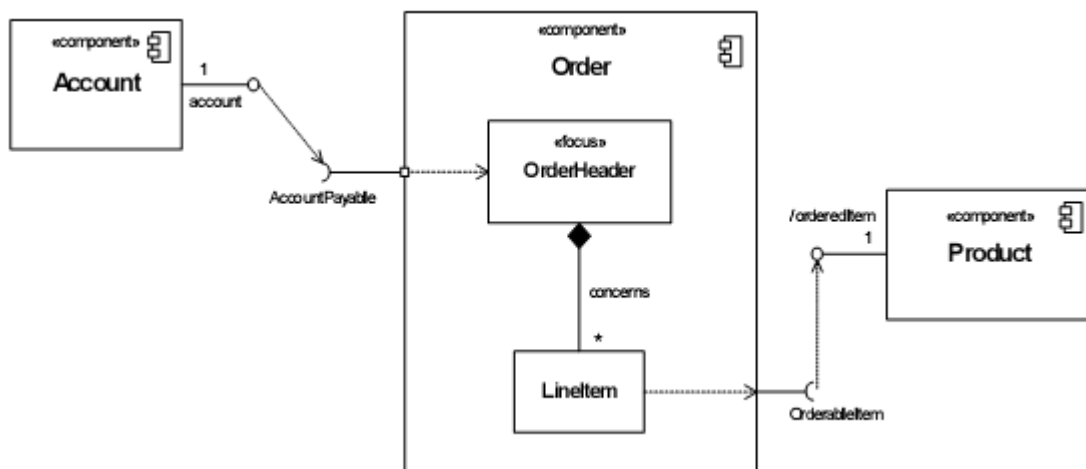


Explicit representation of the provided and required interfaces, allowing interface details such as operation to be displayed (when desired)

Component Diagram

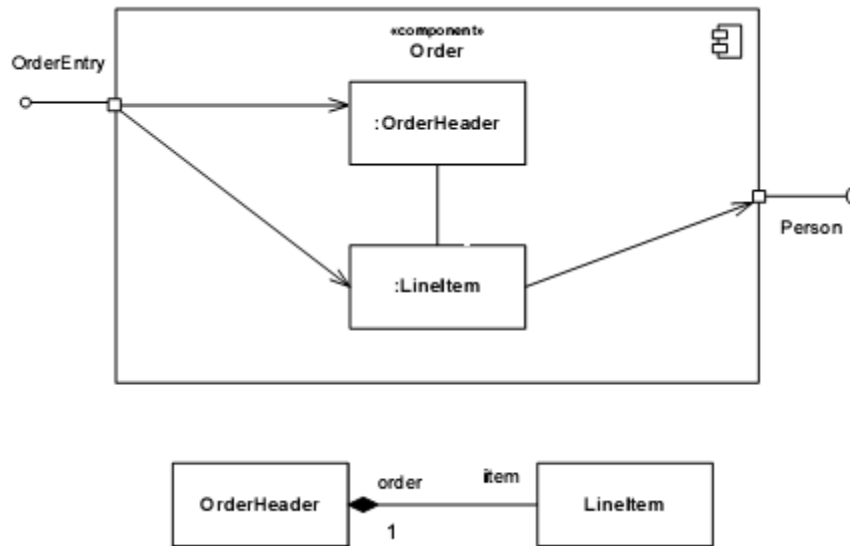


An internal or white-box view of the internal structure of a component that contains other components as parts of its internal assembly



Example of a platform independent model of a component, its provided and required interfaces, and wiring through dependencies on a structure diagram

Component Diagram

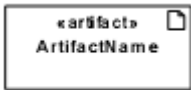
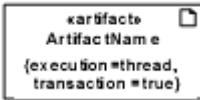
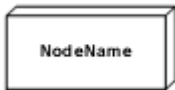
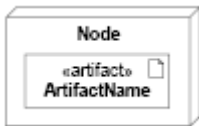
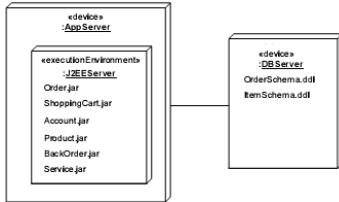


Delegation connectors connect the externally provided interfaces of a component to the parts that realize or require them.

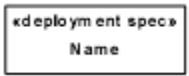
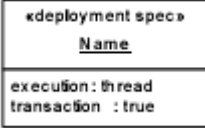
Deployment Diagram

The Deployments package specifies a set of constructs that defines the execution architecture of systems representing the assignment of software artifacts to nodes. Nodes are connected through communication paths to create network systems of arbitrary complexity. Nodes are typically defined in a nested manner, and represent either hardware devices or software execution environments.

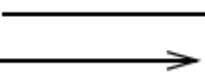
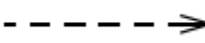
Graphic nodes included in deployment diagrams

Node Type	Notation	Description										
Artifact	  Artifact with annotated deployment properties	An artifact is the specification of a physical piece of information that is used or produced by a software development process, or by deployment and operation of a system. Examples of artifacts include model files, source files, scripts, and binary executable files, a table in a database system, a development deliverable, or a word-processing document, a mail message. UML standard stereotypes that apply to Artifact: <table><tr><td>«document»</td><td>A generic file that is not a «source» file or «executable». Subclass of «file».</td></tr><tr><td>«executable»</td><td>A program file that can be executed on a computer system. Subclass of «file».</td></tr><tr><td>«file»</td><td>A physical file in the context of the system developed.</td></tr><tr><td>«library»</td><td>A static or dynamic library file. Subclass of «file».</td></tr><tr><td>«source»</td><td>A source file that can be compiled into an executable file. Subclass of «file».</td></tr></table>	«document»	A generic file that is not a «source» file or «executable». Subclass of «file».	«executable»	A program file that can be executed on a computer system. Subclass of «file».	«file»	A physical file in the context of the system developed.	«library»	A static or dynamic library file. Subclass of «file».	«source»	A source file that can be compiled into an executable file. Subclass of «file».
«document»	A generic file that is not a «source» file or «executable». Subclass of «file».											
«executable»	A program file that can be executed on a computer system. Subclass of «file».											
«file»	A physical file in the context of the system developed.											
«library»	A static or dynamic library file. Subclass of «file».											
«source»	A source file that can be compiled into an executable file. Subclass of «file».											
Node	  Node with deployed Artifact	A node is computational resource upon which artifacts may be deployed for execution. Nodes can be interconnected through communication paths to define network structures. Dashed arrows with the keyword «deploy» show the capability of a node type to support a component type. Alternatively, this may be shown by nesting component symbols inside the node symbol.										
Device		A Device is a Node, a physical computational resource with processing capability upon which artifacts may be deployed for execution. A device may be a nested element, where a physical machine is decomposed into its elements,										

Deployment Diagram

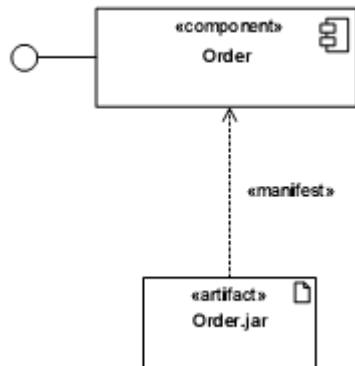
Node Type	Notation	Description
Execution Environment	 An instance of a J2EEServer ExecutionEnvironment	An ExecutionEnvironment is a Node that offers an execution environment for specific types of components that are deployed on it in the form of executable artifacts. ExecutionEnvironments can be nested (e.g., a database ExecutionEnvironment may be nested in an operating system ExecutionEnvironment). Typical examples of standard ExecutionEnvironments that specific profiles might define stereotypes for are «OS», «workflow engine», «database system», and «J2EE container».
Deployment Specification	 DeploymentSpecification with properties  DeploymentSpecification with property values (i.e. an instance)	A deployment specification specifies a set of properties that determine execution parameters of a component artifact that is deployed on a node. DeploymentSpecification for an artifact can be at specification and instance levels.

Graphic paths included in deployment diagrams

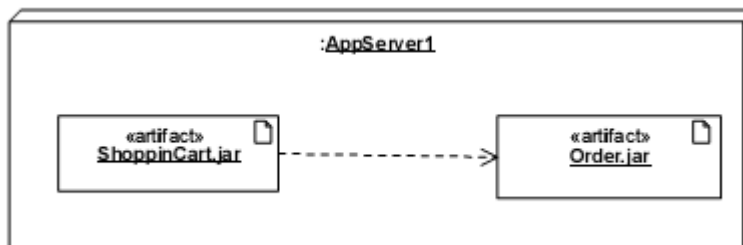
Path Type	Notation	Description
Communication Path		A communication path is an Association that can <i>only</i> be defined between deployment targets, to model the exchange of signals and messages between them.
Dependency		Used to model general dependencies. In Deployment diagrams, this notation is used to depict the following metamodel associations: (i) the relationship between an Artifact and the model element(s) that it implements, and (ii) the deployment of an Artifact (instance) on a Node (instance).

Deployment Diagram

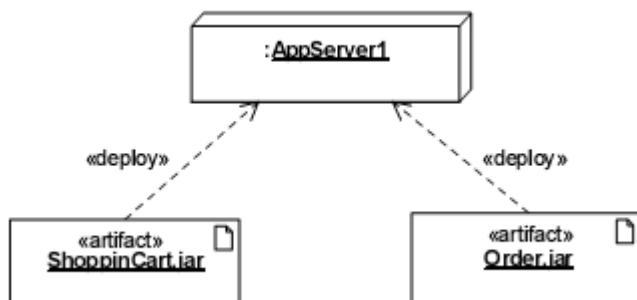
Path Type	Notation	Description
Deployment	«deploy» - - - - - ➔	A deployment is the allocation of an artifact or artifact instance to a deployment target.
Manifestation	«manifest» - - - - - ➔	A manifestation is the concrete physical rendering of one or more model elements by an artifact.



Manifestation relationship between artifacts and components

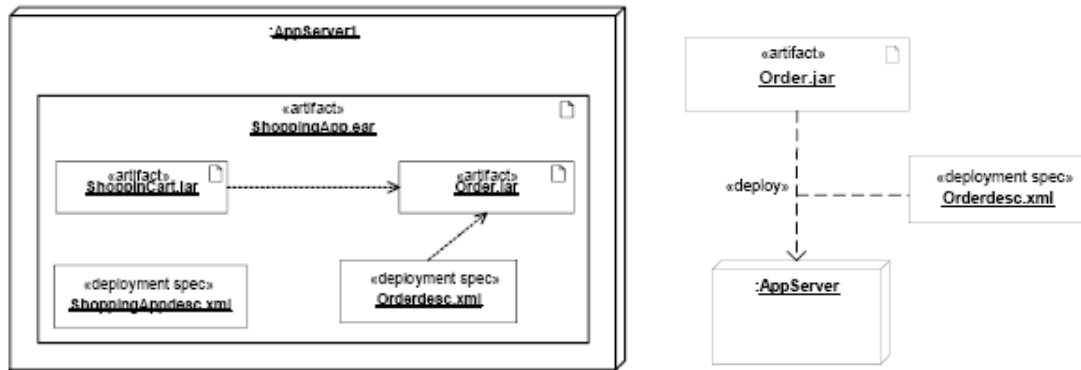


Deployment location of artifacts (including a dependency between the artifacts)

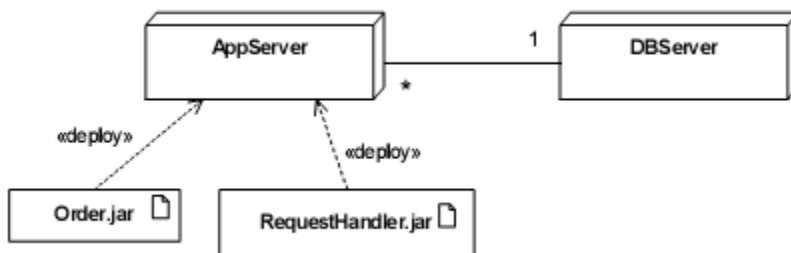


Alternative deployment representation using a dependency called «deploy»

Deployment Diagram



DeploymentSpecification for an artifact

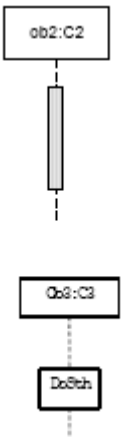
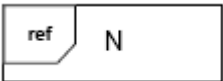


Communication path between two Node types with deployed Artifacts

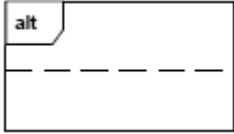
Sequence Diagram

A Sequence Diagram describes an Interaction by focusing on the sequence of Messages that are exchanged, along with their corresponding Occurrence Specifications on the Lifelines.

Graphic nodes included in sequence diagrams

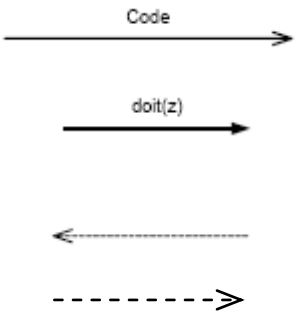
Node Type	Notation	Description
Frame		The notation shows a rectangular frame around the diagram with a name in a compartment in the upper left corner. The keyword sd followed by the Interaction name and parameters is in a pentagon in the upper left corner of the rectangle.
Lifeline		A lifeline represents an individual participant in the Interaction. A Lifeline is shown using a symbol that consists of a rectangle forming its “head” followed by a vertical line (which may be dashed) that represents the lifetime of the participant. The Lifeline head has a shape that is based on the classifier for the part that this lifeline represents. Often the head is a white rectangle containing the name. Information identifying the lifeline is displayed inside the rectangle in the following format: <i>[name] : <class_name></i> (<i>:class_name</i> is mandatory but <i>name</i> is optional).
Execution Specification		An ExecutionSpecification is a specification of the execution of a unit of behavior or action within the Lifeline. The duration of an ExecutionSpecification is represented by the start ExecutionOccurrenceSpecification and the finish ExecutionOccurrenceSpecification.
InteractionUse		It is common to want to share portions of an interaction between several other interactions. An InteractionUse allows multiple interactions to reference an interaction that represents a common portion of their specification. The InteractionUse is shown as a CombinedFragment symbol where the operator is called <i>ref</i>.

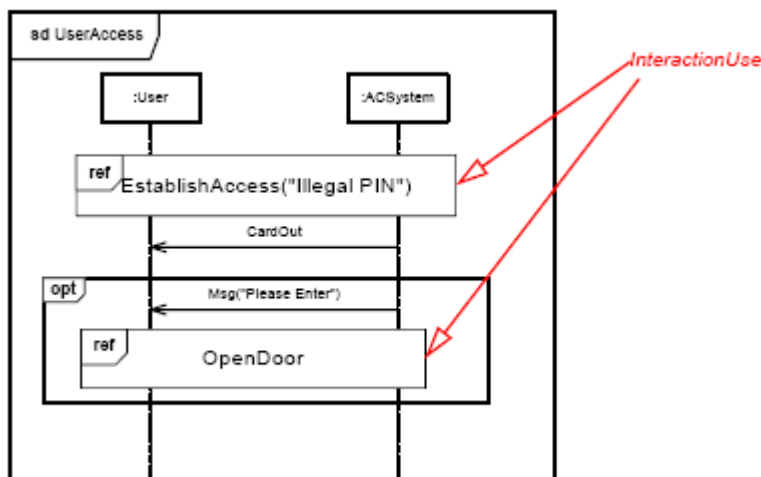
Sequence Diagram

Node Type	Notation	Description
Combined Fragment		A combined fragment is defined by an interaction operator and corresponding interaction operands. Through the use of CombinedFragments the user will be able to describe a number of traces in a compact and concise manner. alt designates that the CombinedFragment represents a choice of behavior. At most one of the operands will be chosen. The chosen operand must have an explicit or implicit guard expression that evaluates to true at this point in the interaction. critical designates that the CombinedFragment represents a critical region, where the traces of the region cannot be interleaved by other OccurrenceSpecifications (on those Lifelines covered by the region). loop operand will iterate minimum the 'minint' number of times (given by the iteration expression in the guard) and at most the 'maxint' number of times. After the minimum number of iterations have executed and the Boolean expression is false the loop will terminate.
Destruction Event		Destruction of the instance described by the lifeline containing the OccurrenceSpecification that references the destruction event.

Sequence Diagram

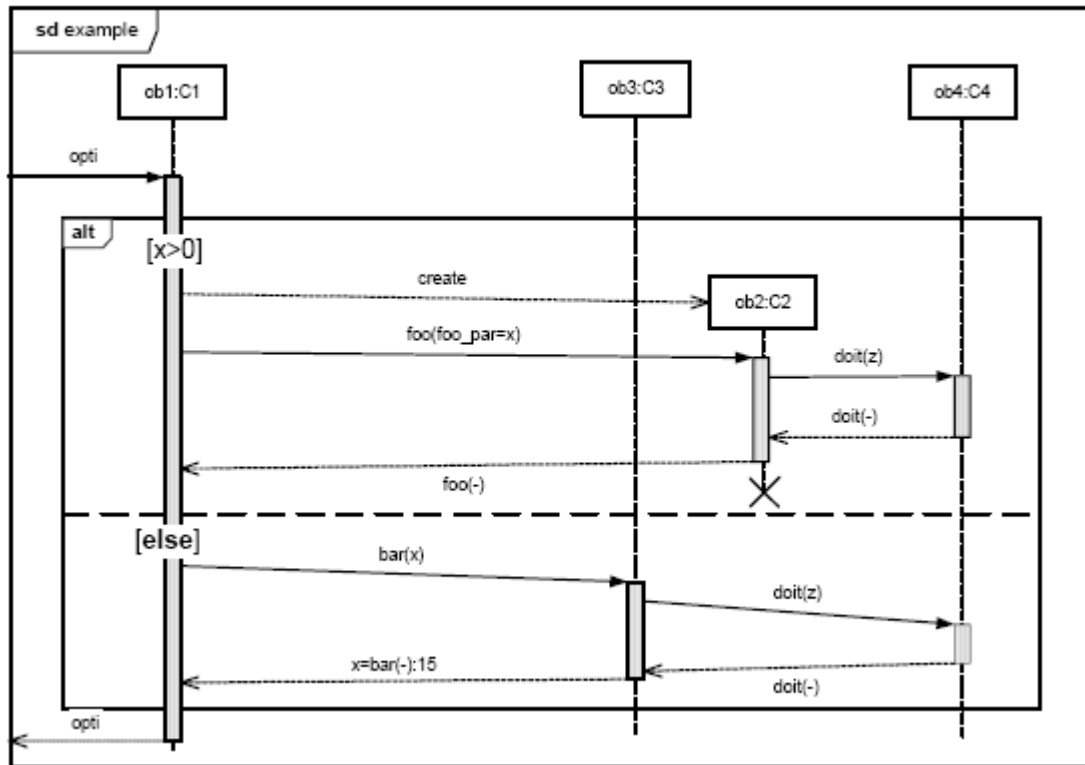
Graphic paths included in sequence diagrams

Path Type	Notation	Description
Message		A Message defines a particular communication between Lifelines of an Interaction. A message is shown as a line from the sender message end to the receiver message end. Messages come in different variants depending on what kind of Message they convey. Here we show an asynchronous message, a call and a reply. These are all <i>complete</i> messages. - Asynchronous Messages have an open arrow head - Synchronous Messages typically represent operation calls and are shown with a filled arrow head. - The reply message from a method has a dashed line. - Object creation Message has a dashed line with an open arrow. Examples of syntax: <i>mymessage(14, - , 3.14, "hello")</i> // second argument is undefined <i>v=mymsg(16, variab):96</i> // this is a reply message carrying the return value 96 assigning it to v <i>mymsg(myint=16)</i> // the input parameter 'myint' is given the argument value 16

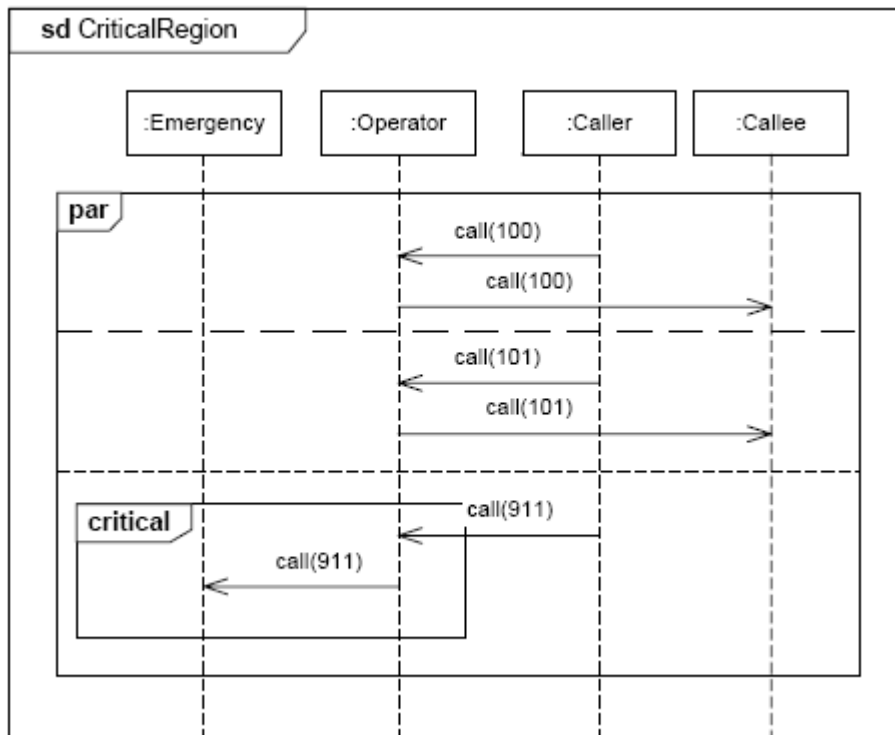


An *InteractionUse* referring the Interaction *EstablishAccess* with (input) argument “Illegal PIN.” Within the optional CombinedFragment there is another *InteractionUse* without arguments referring *OpenDoor*.

Sequence Diagram

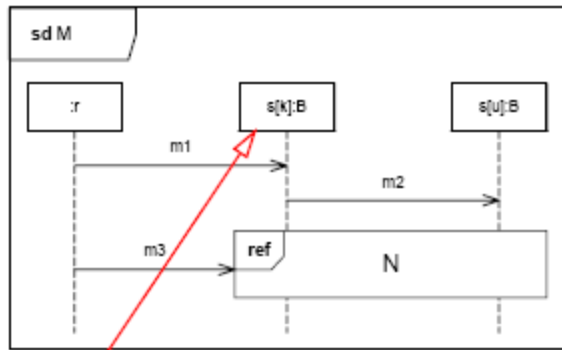


CombinedFragment

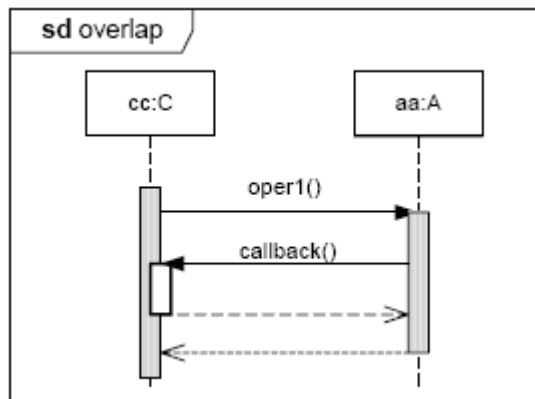
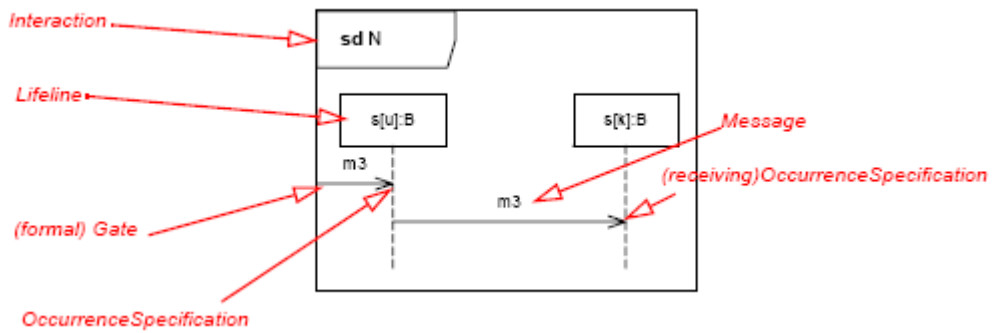


Critical Region - handling of a 911-call must be contiguously handled. The operator must make sure to forward the 911-call before doing anything else. The normal calls, however, can be freely interleaved.

Sequence Diagram



Lifeline



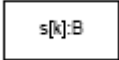
Overlapping execution occurrences

Communication Diagram

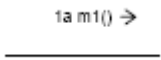
Communication Diagrams focus on the interaction between Lifelines where the architecture of the internal structure and how this corresponds with the message passing is central. The sequencing of Messages is given through a sequence numbering scheme.

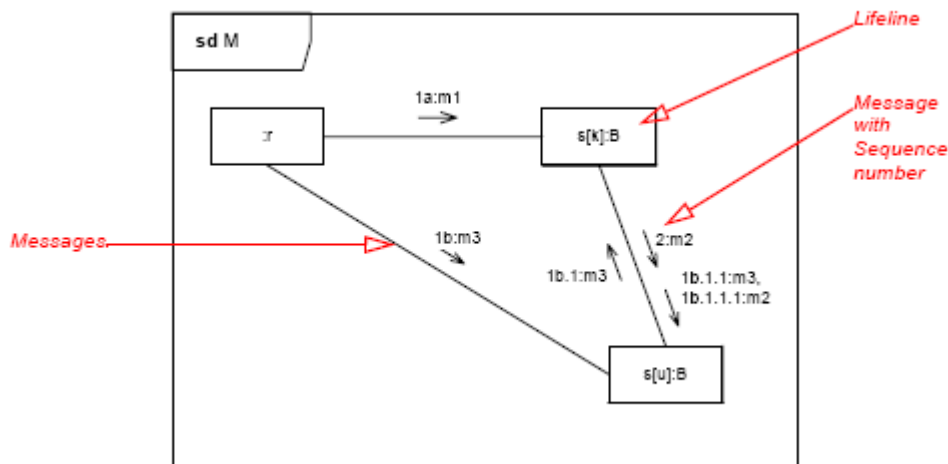
Communication Diagrams correspond to simple Sequence Diagrams that use none of the structuring mechanisms such as InteractionUses and CombinedFragments.

Graphic nodes included in communication diagrams

Node Type	Notation	Description
Frame		The same as for the Sequence Diagram.
Lifeline		The same as for the Sequence Diagram, but without the vertical line that represents the lifetime of the participant.

Graphic paths included in communication diagrams

Path Type	Notation	Description
Message		The arrow shown here indicates the communication direction. On Communication Diagrams, the Messages are decorated by a small arrow in the direction of the Message close to the Message name and sequence number along the line between the lifelines.



Activity Diagram

Activity modeling emphasizes the sequence and conditions for coordinating lower-level behaviors.

An activity specifies the coordination of executions of subordinate behaviors, using a control and data flow model. The flow of execution is modeled as activity nodes connected by activity edges.

Each action in an activity may execute zero, one, or more times for each activity execution... The activities specification (at the higher compliance levels) allows for several (logical) threads of control executing at once and synchronization mechanisms to ensure that activities execute in a specified order.

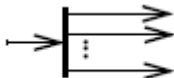
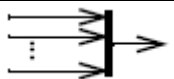
Graphic nodes included in activity diagrams

There are 3 kinds of ActivityNodes: ActionNode, ObjectNode and ControlNode



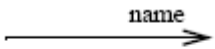
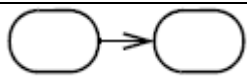
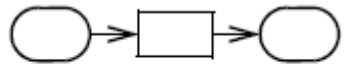
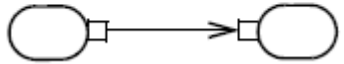
Node Type	Notation	Description
Action		An action represents a single step within an activity, that is, one that is not further decomposed within the activity. The execution of an action represents some transformation or processing in the modeled system, be it a computer system or otherwise. An activity represents a behavior that is composed of individual elements that are actions.
ActivityFinal		An activity final node is a final node that stops all flows in an activity. An activity may have more than one activity final node. The first one reached stops all flows in the activity.
DecisionNode		A decision node is a control node that chooses between outgoing flows. A decision node has one incoming edge and multiple outgoing activity edges.
FlowFinal		A flow final node is a final node that terminates a flow. A flow final destroys all tokens that arrive at it. It has no effect on other flows in the activity.

Activity Diagram

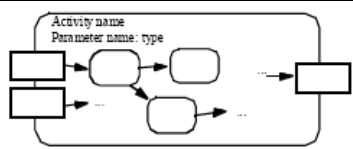
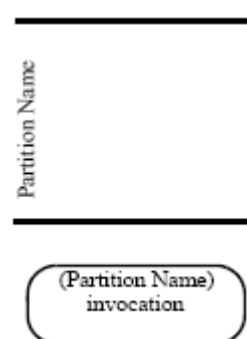
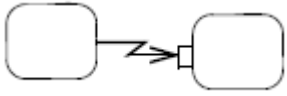
Node Type	Notation	Description
ForkNode		A fork node is a control node that splits a flow into multiple concurrent flows. A fork node has one incoming edge and multiple outgoing edges. Fork nodes are introduced to support parallelism in activities.
InitialNode		An initial node is a control node at which flow starts when the activity is invoked. Note that flows can also start at other nodes, e.g. in case of AcceptEventAction, so initial nodes are not required for an activity to start execution. If an activity has more than one initial node, then invoking the activity starts multiple flows, one at each initial node.
JoinNode		A join node is a control node that synchronizes multiple flows. The join node must have one or more activity edges entering it, and only one edge leaving it.
MergeNode		A merge node is a control node that brings together multiple alternate flows. It is not used to synchronize concurrent flows but to accept one among several alternate flows. A merge node has multiple incoming edges and a single outgoing edge. Merge nodes are introduced to support bringing multiple flows together in activities.
AcceptEventAction		AcceptEventAction is an action that waits for the occurrence of an event meeting specified condition.
SendSignalAction		SendSignalAction is an action that creates a signal instance from its inputs, and transmits it to the target object, where it may cause the firing of a state machine transition or the execution of an activity.

Activity Diagram

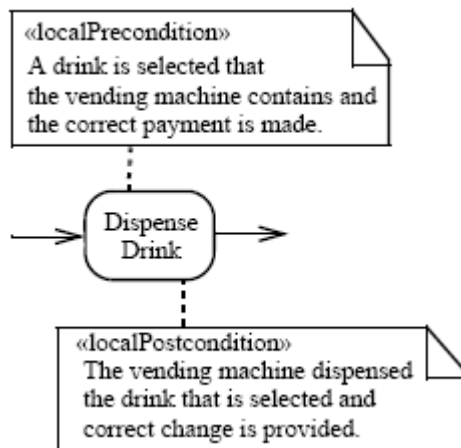
Graphic paths included in activity diagrams

Path Type	Notation	Description
ActivityEdge	 <i>Regular activity edge</i>  <i>Activity edge with name</i>  (where, n is connector name)	ActivityEdge is an abstract class for the connections along which tokens flow between activity nodes. It covers control and data flow edges. Activity edges are directed connections, that is, they have a source and a target, along which tokens may flow.
ControlFlow		A control flow is an edge that starts an activity node after the previous one is finished. It passes control tokens. Tokens offered by the source node are all offered to the target node.
ObjectFlow	 	An object flow is an activity edge that can have objects or data passing along it.

Graphic elements for containment in activity diagrams

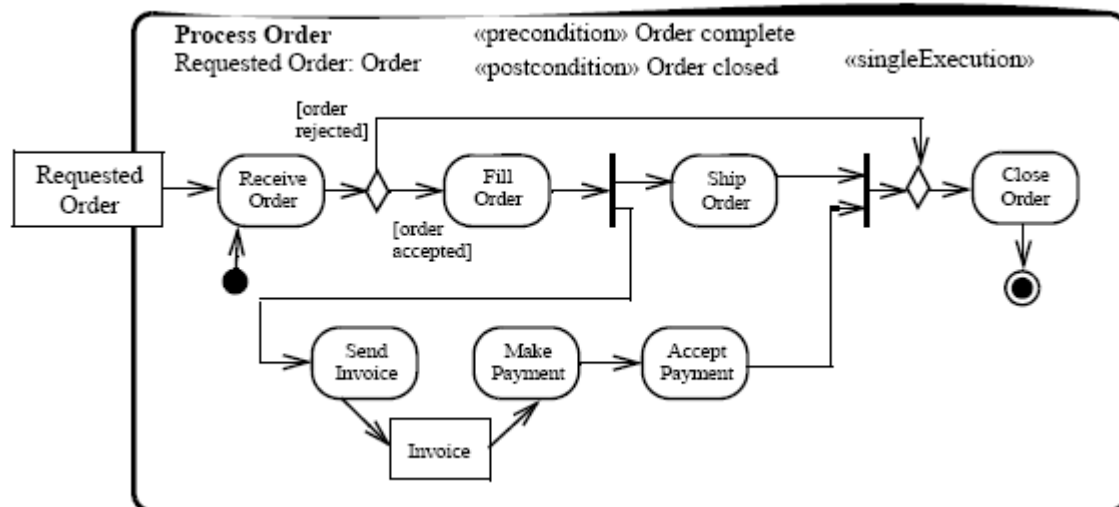
Path Type	Notation	Description
Activity		
ActivityPartition		Partitions divide the nodes and edges to constrain and show a view of the contained nodes. Partitions can share contents. They often correspond to organizational units in a business model. They may be used to allocate characteristics or resources among the nodes of an activity.
ExceptionHandler		An exception handler is an element that specifies a body to execute in case the specified exception occurs during the execution of the protected node.

Activity Diagram

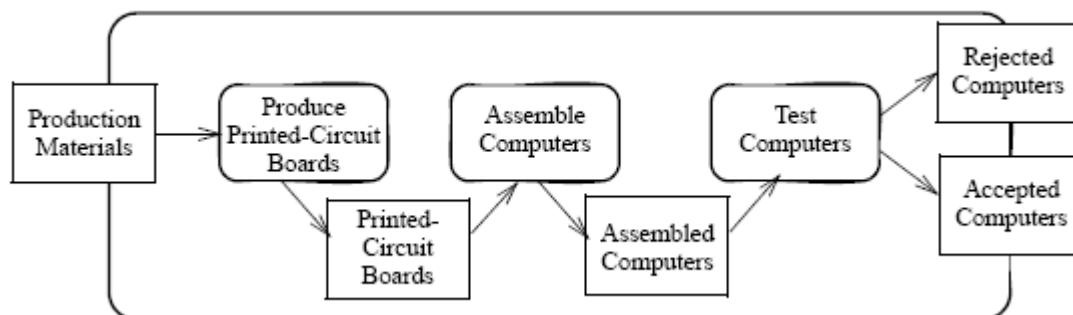


Example of an action with local pre- and post-conditions

The definition of Process Order below uses the border notation to indicate that it is an activity. It has pre- and post conditions on the order (see Behavior). All invocations of it use the same execution.

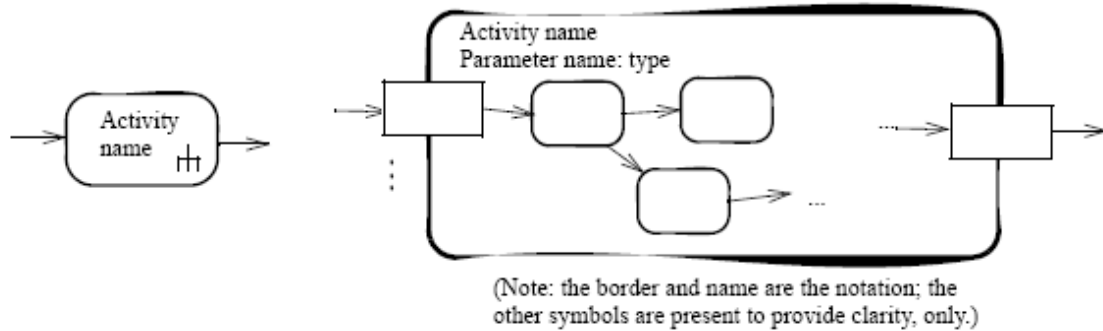


Example of an activity with input parameter

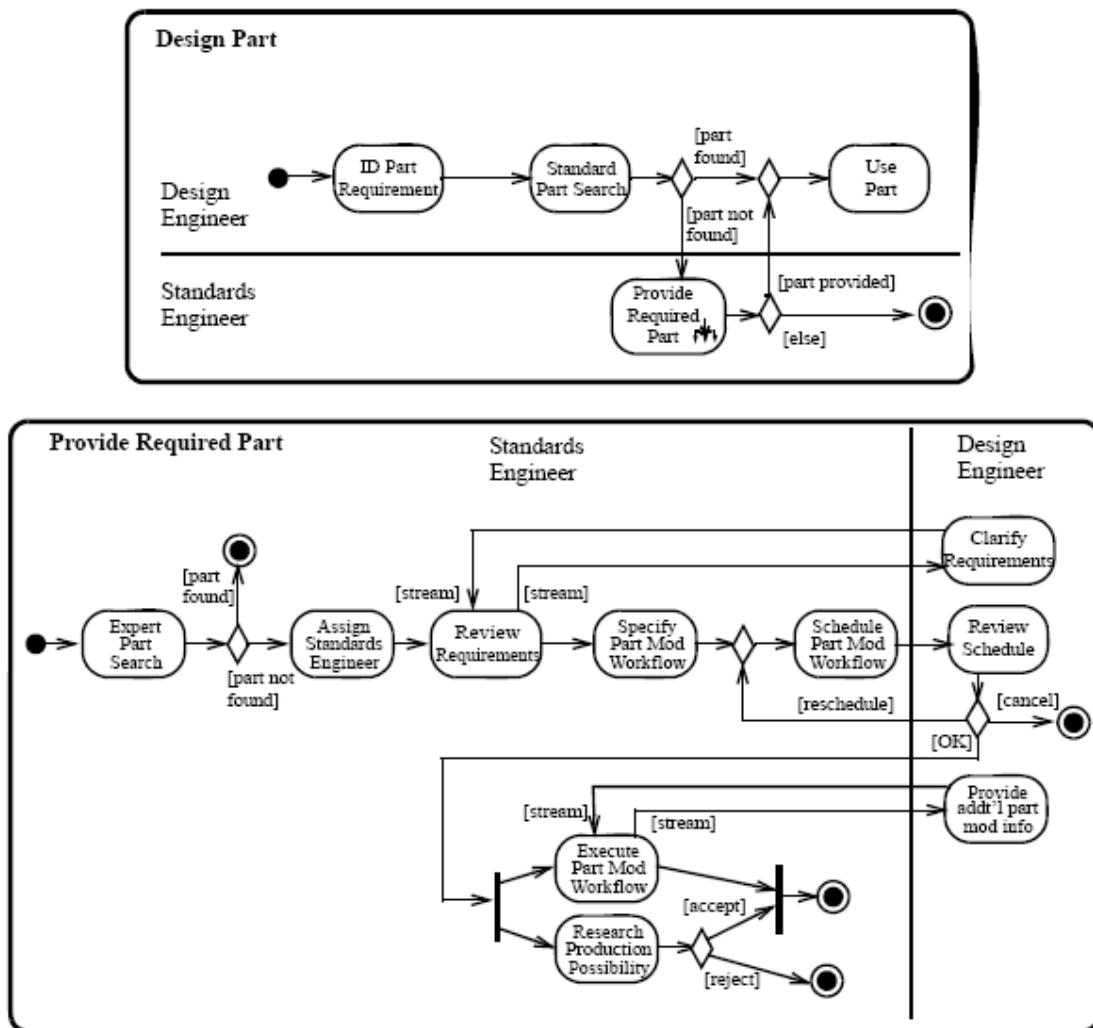


Example of ObjectNode

Activity Diagram

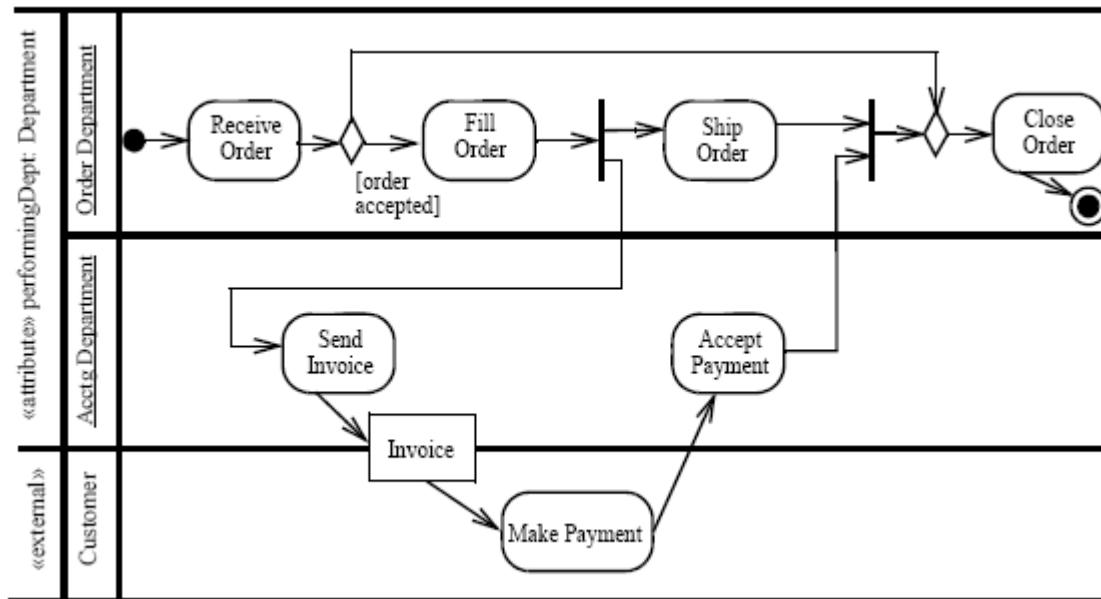


Call of an activity is indicated by placing a rake-style symbol within the symbol. An alternative notation in the case of an invoked activity is to show the contents of the invoked activity inside a large round-cornered rectangle. Edges flowing into the invocation connect to the parameter object nodes in the invoked activity. The parameter object nodes are shown on the border of the invoked activity.

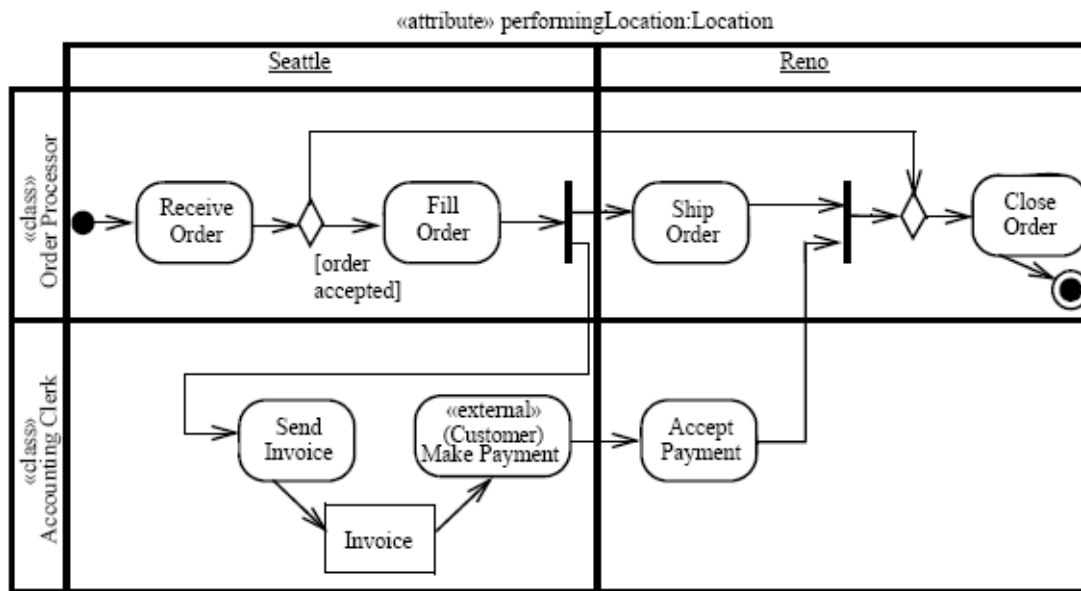


Activity Diagram

Workflow example, where Design Part calls the Provide Required Part Activity

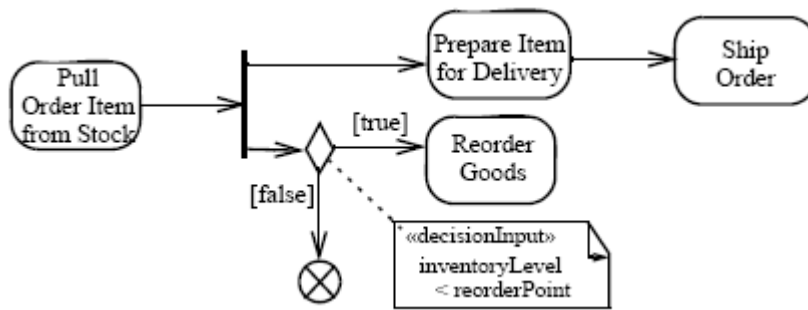


Activity partition using swimlane example

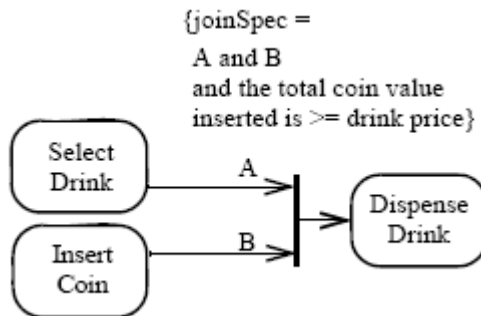


Activity partition using multidimensional swimlane example

Activity Diagram

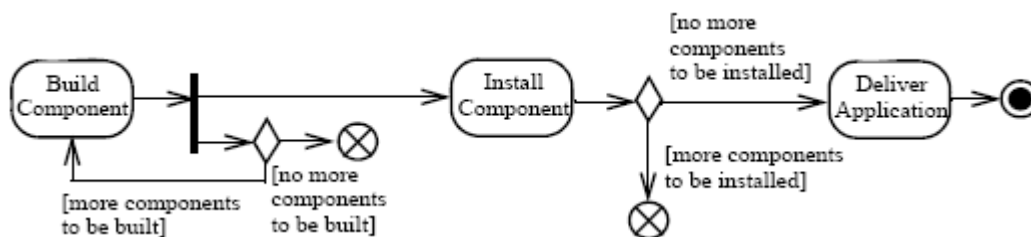


Decision node example



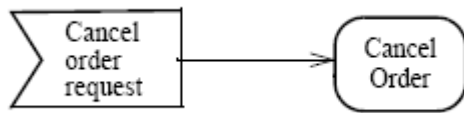
Join node example

The figure below illustrates two kinds of final node: flow final and activity final. In this example, it is assumed that many components can be built and installed before finally delivering the resulting application. Here, the Build Component behavior occurs iteratively for each component. When the last component is built, the end of the building iteration is indicated with a flow final. However, even though all component building has come to an end, other behaviors are still executing. When the last component has been installed, the application is delivered. When Deliver Application has completed, control is passed to an activity final node—indicating that all processing in the activity is terminated.

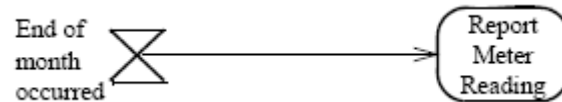


Flow final and activity final example.

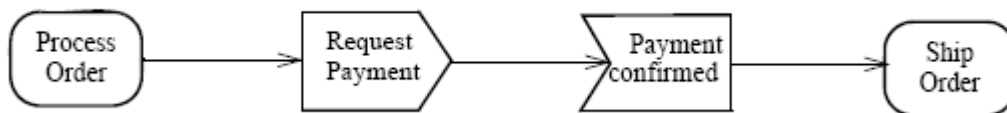
Activity Diagram



The acceptance of a signal indicating the cancellation of an order causes an invocation of a cancellation behavior.

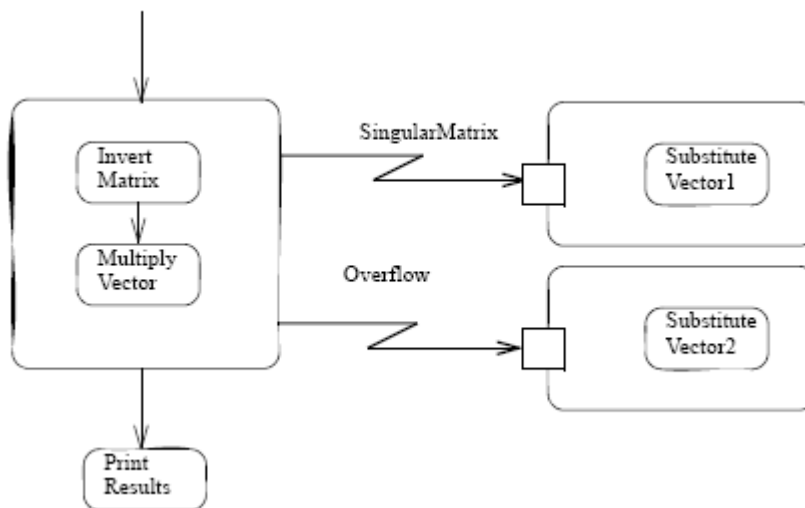


The end-of-month accept time event action generates an output at the end of the month.



A request payment signal is sent after an order is processed. The activity then waits to receive a payment confirmed signal. Acceptance of the payment confirmed signal is enabled only after the request for payment is sent; no confirmation is accepted until then. When the confirmation is received, the order is shipped.

AcceptEventAction and SendSignalAction example



Exception Handler example

State Machine Diagram

A state machine diagram is a graph that represents a state machine.

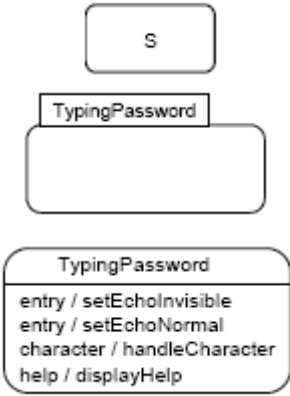
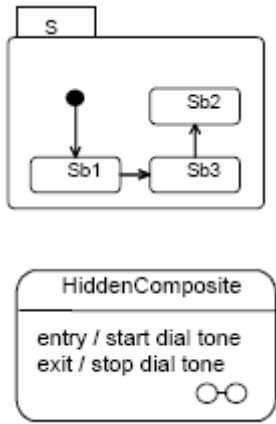
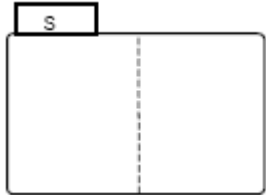
Event occurrences are detected, dispatched, and then processed by the state machine, one at a time. The order of dequeuing is not defined, leaving open the possibility of modeling different priority-based schemes.

The processing of a single event occurrence by a state machine is known as a *run-to-completion step*. When an event occurrence is detected and dispatched, it may result in one or more transitions being enabled for firing. In the presence of orthogonal regions it is possible to fire multiple transitions as a result of the same event occurrence.

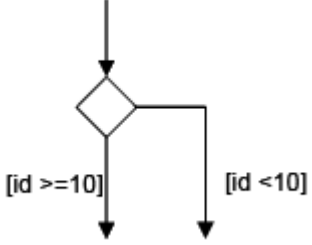
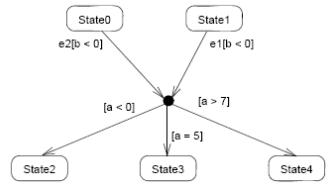
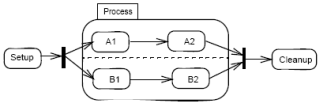
Graphic nodes included in state machine diagrams

Node Type	Notation	Description
State	See Simple State and Composite State	The compartments of a state are: • name compartment - holds the (optional) name of the state • internal activities compartment - holds a list of internal actions or state (do) activities (behaviors) that are performed while the element is in the state • <i>entry</i> — An optional behavior that is executed whenever this state is entered regardless of the transition taken to reach the state. If defined, entry actions are always executed to completion prior to any internal behavior or transitions performed within the state. • <i>exit</i> — An optional behavior that is executed whenever this state is exited regardless of which transition was taken out of the state. If defined, exit actions are always executed to completion only after all internal activities and transition actions have completed execution. • <i>do</i> — An optional behavior that is executed while being in the state. The execution starts when this state is entered following the entry behaviour (if any), and stops either by itself or when the state is exited whichever comes first. • internal transitions compartment A composite state has in addition a decomposition compartment which shows its composition structure in terms of regions, states, and transition in a nested diagram.

State Machine Diagram

Node Type	Notation	Description
Simple state		A state that does not have substates (i.e., it has no regions and it has no submachine state machine).
Composite state		Either contains one region or is decomposed into two or more orthogonal <i>regions</i>. Each region has a set of mutually exclusive disjoint subvertices and a set of transitions. Each region of a composite state may have an initial pseudostate and a final state. A transition to the enclosing state represents a transition to the initial pseudostate in each region. A transition to a final state represents the completion of behavior in the enclosing region. Completion of behavior in all orthogonal regions represents completion of behavior by the enclosing state and triggers a completion event on the enclosing state. Completion of the topmost regions of an object corresponds to its termination. An entry pseudostate is used to join an external transition terminating on that entry point to an internal transition emanating from that entry point. An exit pseudostate is used to join an internal transition terminating on that exit point to an external transition emanating from that exit point.
Region		A region is an orthogonal part of either a composite state or a state machine. It contains states and transitions. A region can have at most one initial vertex. A region can have at most one deep history vertex. A region can have at most one shallow history vertex.
Final state		A special kind of state signifying that the enclosing region is completed. If the enclosing region is directly contained in a state machine and all other regions in the state machine also are completed, then it means that the entire state machine is completed.

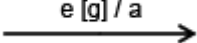
State Machine Diagram

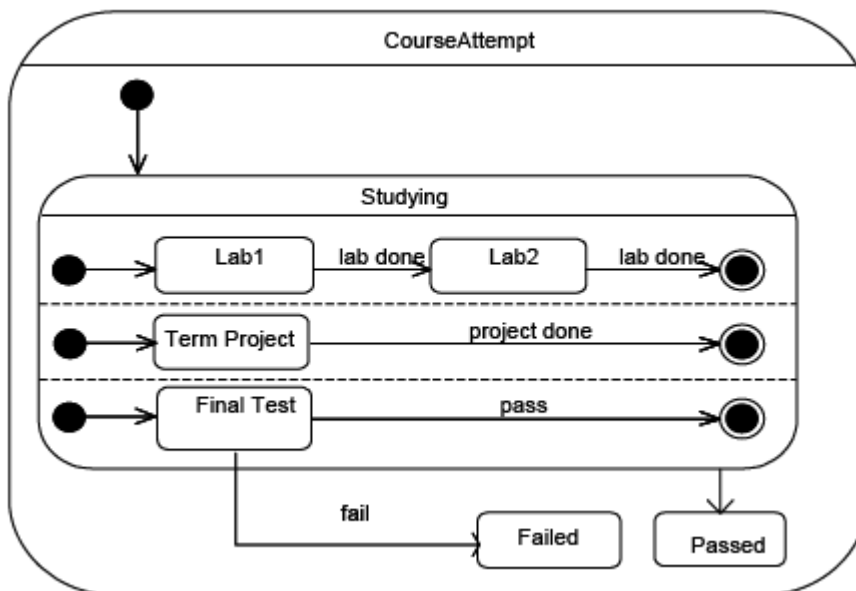
Node Type	Notation	Description
Pseudostate		An abstraction that encompasses different types of transient vertices in the state machine graph, typically used to connect multiple transitions into more complex state transitions paths.
Initial Pseudostate		Source for a single transition to the <i>default</i> state of a composite state. There can be at most one initial vertex in a region.
Choice Pseudostate		Realizes a <i>dynamic conditional branch</i> – when reached, results in the dynamic evaluation of the guards of the triggers of its outgoing transitions.
Junction Pseudostate		Realizes a <i>static conditional branch</i> - can be used to converge multiple incoming transitions into a single outgoing transition representing a shared transition path, or to split an incoming transition into multiple outgoing transition segments with different guard conditions.
Fork Pseudostate		Split an incoming transition into two or more transitions terminating on orthogonal target vertices (i.e., vertices in different orthogonal regions of a composite state). The segments outgoing from a fork vertex must not have guards or triggers.
Join Pseudostate		Merges several transitions emanating from source vertices in different orthogonal regions. The transitions entering a join vertex cannot have guards or triggers.
Entry Point Pseudostate	again 	An entry point of a state machine or composite state. In each region of the state machine or composite state it has a single transition to a vertex within the same region.
Exit Point Pseudostate	aborted 	Implies the exit of composite state and the triggering of the transition that has this exit point as source in the composite state.

State Machine Diagram

Node Type	Notation	Description
Deep History Pseudostate		Represents the most recent active configuration of the composite state that directly contains this pseudostate (e.g., the state configuration that was active when the composite state was last exited). A composite state can have at most one deep history vertex. At most one transition may originate from the history connector to the <i>default</i> deep history state.
Shallow History Pseudostate		A transition coming into the shallow history vertex is equivalent to a transition coming into the most recent active substate of a state. At most one transition may originate from the history connector to the <i>default</i> shallow history state.

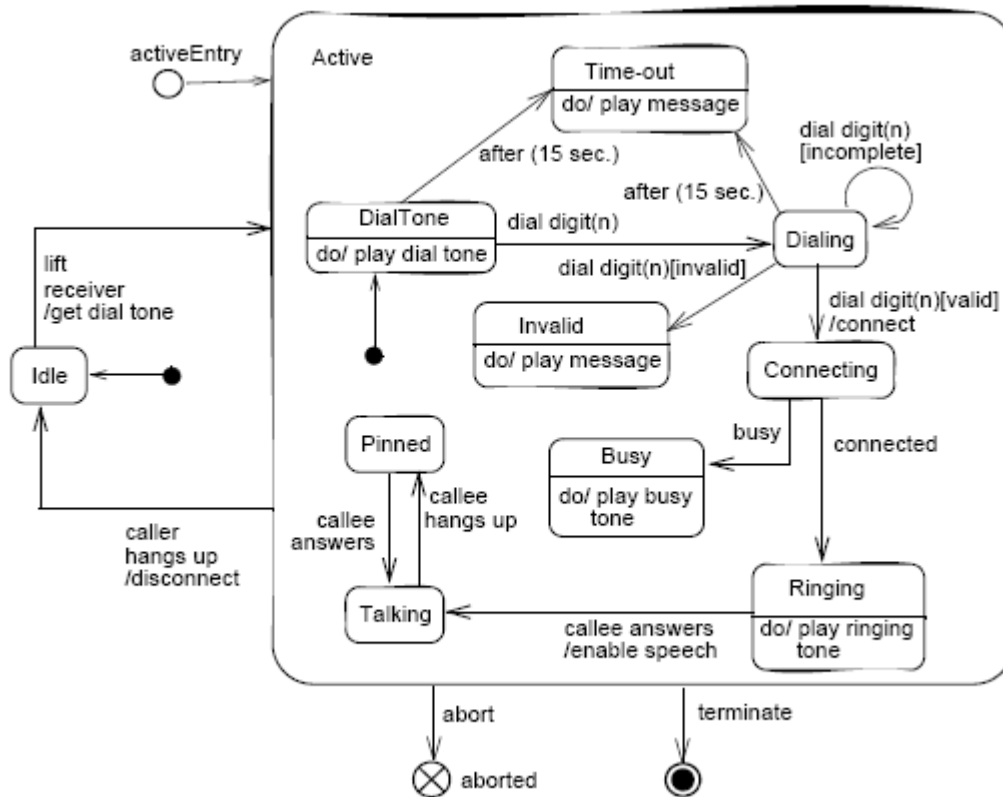
Graphic paths included in state machine diagrams

Path Type	Notation	Description
Transition		A directed relationship between a source vertex and a target vertex e - Event / Trigger g – Guard, evaluated to true before the transition can be triggered a – Activity or behaviour executed during transition



Orthogonal state with regions

State Machine Diagram



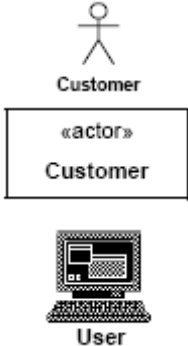
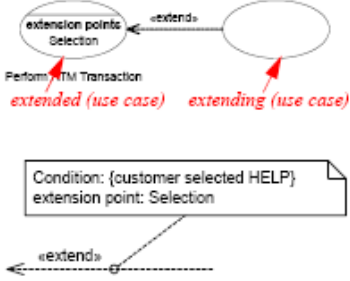
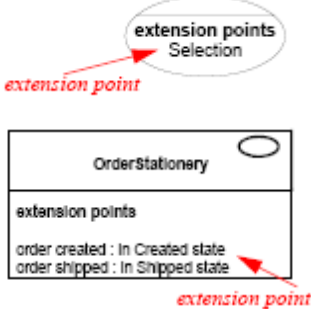
Statemachine diagram for the state machine for simple telephone object

Use Case Diagram

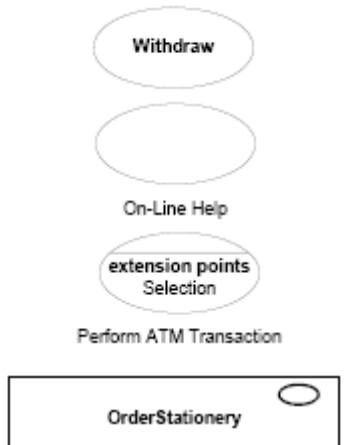
Use cases are a means for specifying required usages of a system. Typically, they are used to capture the requirements of a system, that is, what a system is supposed to do.

The key concepts associated with use cases are *actors*, *use cases*, and the *subject*. The subject is the system under consideration to which the use cases apply. The users and any other systems that may interact with the subject are represented as actors. Actors always model entities that are outside the system. The required behavior of the subject is specified by one or more use cases, which are defined according to the needs of actors.

Graphic nodes included in use case diagrams

Node Type	Notation	Description
Actor		An Actor models a type of role played by an entity that interacts with the subject (e.g., by exchanging signals and data), but which is <i>external</i> to the subject. Actors may represent roles played by human users, external hardware, or other subjects. An actor can only have associations to use cases, components, and classes. An actor must have a name.
Extend		Specifies how and when the behavior defined in the extending use case can be inserted into the behavior defined in the extended use case. The extension takes place at one or more specific extension points defined in the extended use case. Extend with condition
Extension Point		An ExtensionPoint is a feature of a use case that identifies a point where the behavior of a use case can be augmented with elements of another (extending) use case. An ExtensionPoint must have a name.

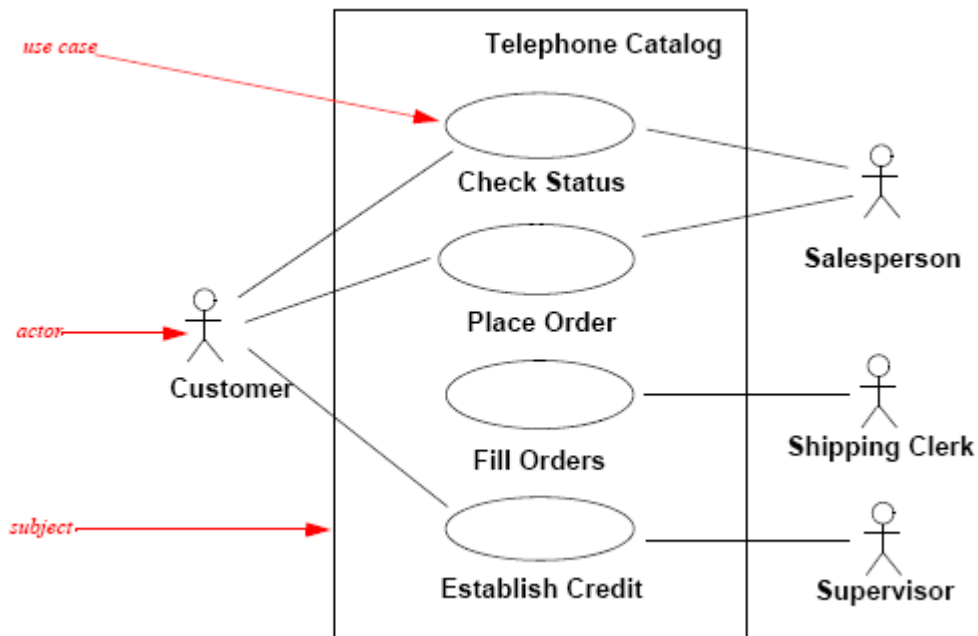
Use Case Diagram

Node Type	Notation	Description
Include		The behavior defined in the including use case is included in the behavior of the base use case. The include relationship is intended to be used when there are common parts of the behavior of two or more use cases. This common part is then extracted to a separate use case, to be included by all the base use cases having this part in common. The included use case is not optional, and is always required for the including use case to execute correctly.
Use Case		A use case is the specification of a unit of functionality performed by a system, which yields an observable result that is of value for one or more actors or other stakeholders of the system.

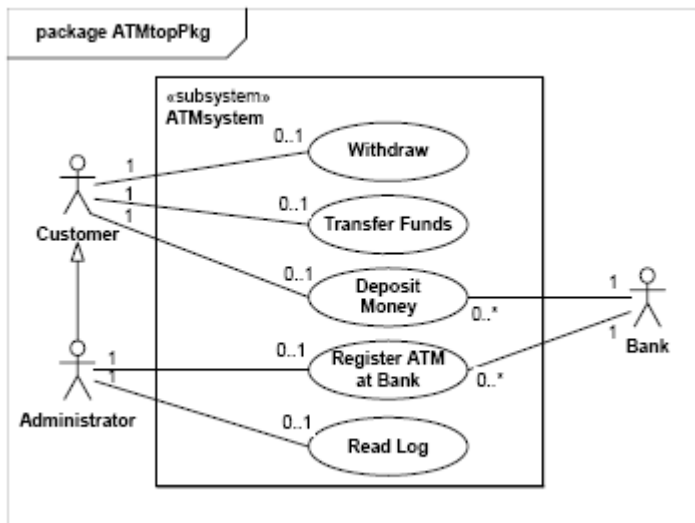
Graphic paths included in use case diagrams

Path Type	Notation	Description
Association		Same as for Class Diagram In Use Case Diagram, the Association connects Actors with the Use Cases in which they participate.

Use Case Diagram

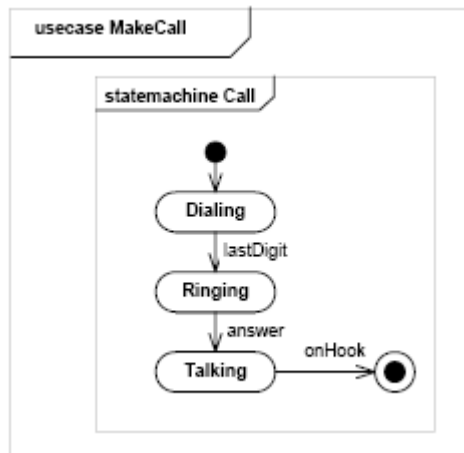


UseCase diagram with a rectangle representing the boundary of the subject

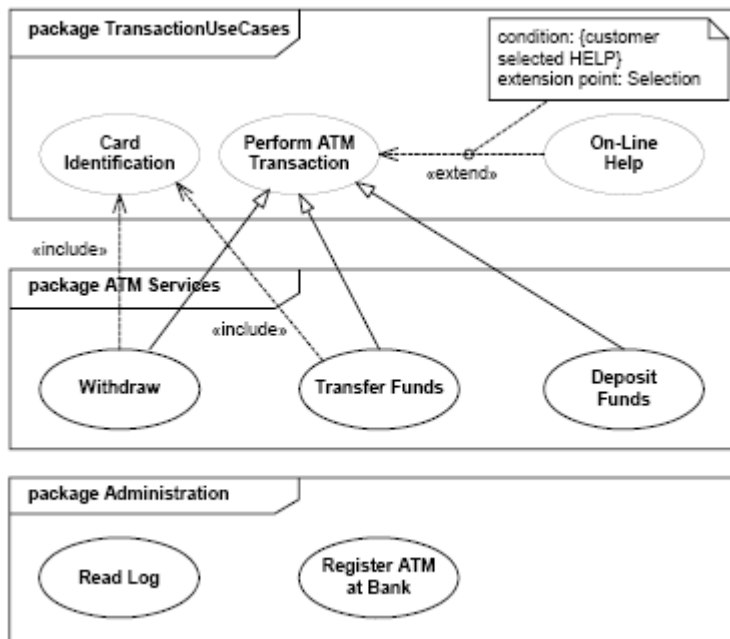


Use cases owned by a package - A use case may be owned either by a package or by a classifier (typically the classifier specifying the subject).

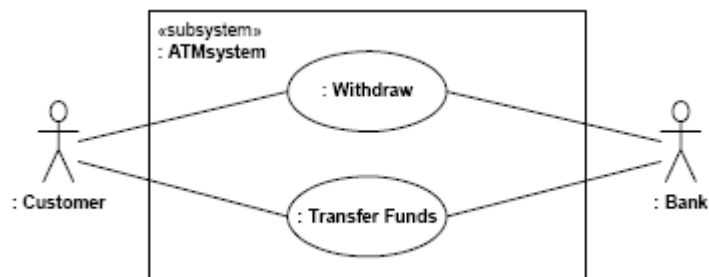
Use Case Diagram



Use case with an associated state machine behavior



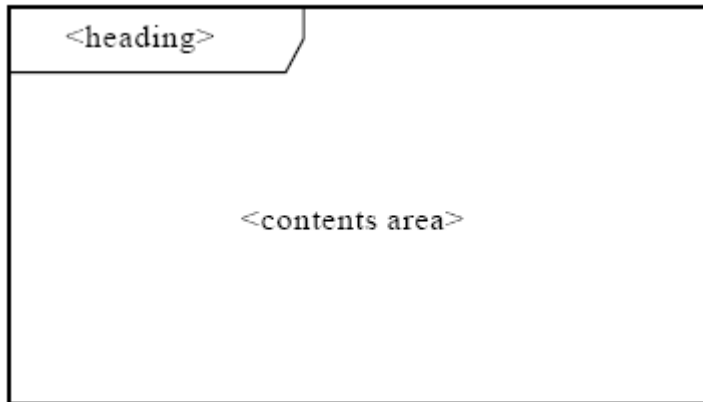
Use cases owned by various packages



UML Diagrams

UML diagrams contain graphical elements (nodes connected by paths) that represent elements in the UML model.

Each diagram has a *contents area*. As an option, it may have a *frame* and a *heading* as shown:-



The frame is a rectangle. In cases where not needed, the frame may be omitted and implied by the border of the diagram area provided by a tool. In case the frame is omitted, the heading is also omitted.

The diagram contents area contains the graphical symbols; the primary graphical symbols define the type of the diagram.

The heading is a string contained in a name tag (rectangle with cutoff corner) in the upper leftmost corner of the rectangle, with the following syntax: `[<kind>]<name>[<parameters>]` The heading of a diagram represents the kind, name, and parameters of the namespace enclosing or the model element owning elements that are represented by symbols in the contents area.

UML diagrams may have the following kinds of frame names as part of the heading:

- activity
- class
- component
- deployment
- interaction
- package
- state machine
- use case

In addition to the long form names for diagram heading types, the following abbreviated forms can also be used:

- **act** (for activity frames)
- **cmp** (for component frames)
- **dep** (for deployment frames)
- **sd** (for interaction frames)
- **pkg** (for package frames)
- **stm** (for state machine frames)
- **uc** (for use case frames)